

Multi-Class Traffic-Aware Deep Reinforcement Learning for Active Queue Management

A Deep Q-Network (DQN) Approach to Class-Aware Congestion Control

NS-3 Simulation Project Report

**Wireless 802.11 (Mobile) and Wireless 802.15.4 (Static)
Topologies**

Parameter Sweep Evaluation with DQN, PfifoFast, CoDel, RED, and PIE

Ruwad Naswan

Student ID: **2105051**

Simulator: **ns-3.42**

DQN Library: **tiny-dnn v1.0.0a3**

Topologies: **WiFi 802.11n Mobile + WPAN 802.15.4 Static**

Seeds per experiment: **3**

Simulation time: **30 seconds per run** April 8, 2026

Contents

1	Introduction	3
2	Network Topologies Under Simulation	4
2.1	Wireless 802.11n Mobile Topology	4
2.1.1	Architecture	4
2.1.2	Channel Model	5
2.1.3	Mobility Model	5
2.1.4	Traffic Model	5
2.1.5	Energy Model	5
2.2	Wireless 802.15.4 Static Topology	6
2.2.1	Architecture	6
2.2.2	Coverage Area and Path Loss Model	7
2.2.3	Traffic Model	7
2.2.4	Energy Model	7
3	Parameters Under Variation	7
3.1	WiFi 802.11n Mobile — Parameter Sweeps	8
3.2	WPAN 802.15.4 Static — Parameter Sweeps	8
4	Modifications Made in the Simulator	8
4.1	DQN Queue Disc Module	8
4.1.1	State Space (18 dimensions)	8
4.1.2	Action Space (5 actions)	9
4.1.3	Neural Network Architecture	9
4.1.4	Double DQN	9
4.1.5	Action Masking	9
4.1.6	Reward Function (v5 — Byte-Weighted, 7 Components)	10
4.2	Topology Files	10
4.3	Build Fixes	10
5	Results with Graphs	10
5.1	WiFi 802.11n Mobile Topology	11
5.1.1	Throughput	11
5.1.2	End-to-End Delay	12
5.1.3	Packet Delivery Ratio and Drop Ratio	13
5.1.4	Jitter	14
5.1.5	QoS Composite Score	14
5.1.6	Energy Consumption	15
5.2	WPAN 802.15.4 Static Topology	16
5.2.1	Throughput and Delay	16
5.2.2	nNodes Sweep: Traffic Density Effect	17
5.2.3	Coverage Area Sweep	17
5.2.4	Jitter and QoS Score	18
5.2.5	Energy Consumption	19
6	The DQN-AQM Algorithm: Design and Evolution	19
6.1	Overview	19

6.2	Reward Function Evolution	20
6.3	Training Results	21
7	Summary Findings	23
7.1	WiFi 802.11n Mobile — Summary	23
7.2	WPAN 802.15.4 Static — Summary	23
7.3	Multi-Seed Validation (3 Mbps Baseline)	24
7.4	Varied Topology Generalization	25
7.5	Heavy Congestion Scenario (1 Mbps)	26
7.6	Non-Stationary Traffic Adaptation	27
8	Discussion and Reflection on Findings	28
8.1	DQN vs Classical AQMs: When Does Each Win?	28
8.2	Why Energy is Identical Across AQMs	28
8.3	The Fairness-Delay Tradeoff	28
8.4	Reward Hacking: A Practical Lesson	28
8.5	Bandwidth Generalization	29
8.6	Limitations and Future Work	29
9	Conclusion	29

1 Introduction

Active Queue Management (AQM) is a fundamental mechanism in modern network infrastructure responsible for managing congestion at bottleneck links. While classical AQMs such as RED [1], CoDel [2], PIE [3], and PfifoFast make drop or mark decisions based solely on queue length or sojourn time, they are fundamentally *class-blind* — they cannot distinguish between a latency-sensitive video stream, a bulk TCP file transfer, a tiny IoT sensor packet, or a bursty web request.

This project implements and evaluates a **Multi-Class Traffic-Aware Deep Q-Network (DQN) for Active Queue Management** in ns-3.42. The work extends the baseline DQN-AQM proposed by Kim [6], which introduced the first deep learning based AQM scheme for IoT fog/edge networks. The base paper uses a 3-dimensional state $\{L_t, R_{\text{deq},t}, d_t\}$ (queue length, dequeue rate, queuing delay) and a binary drop/no-drop action with a 2-component reward clipped to $[-1, +1]$. While it outperforms classical AQMs on delay and jitter, it is *class-blind*: all IoT device traffic is treated identically regardless of protocol or application type.

Our agent observes per-class queue metrics and selects *which traffic class to drop* at each congestion event, rather than making a binary drop/no-drop decision. The core hypothesis is that class-aware dropping — targeting traffic whose loss has the smallest network-wide cost — outperforms class-blind policies under congestion.

The implementation extends the base paper’s single-class DQN-AQM with:

- An 18-dimensional per-class state space (queue occupancy, per-class delay, arrival rates, loss rates, queue shares)
- A 5-action space (drop Video, drop File, drop IoT, drop Web, no-drop)
- Double DQN architecture to reduce Q-value overestimation
- Absolute action masking (hard fairness constraints)
- A 7-component byte-weighted reward function (v5 design)

The AQM is evaluated across two wireless topologies as per the course assignment requirements:

1. **Wireless 802.11n Mobile Topology:** A WiFi access point with mobile stations using the RandomWaypoint mobility model, connected to a remote server via a 2 Mbps P2P bottleneck link where the DQN-AQM is installed.
2. **Wireless 802.15.4 Static Topology:** An IoT-style sensor network emulated using CSMA at 250 Kbps (802.15.4 standard data rate), with static sensor nodes feeding into a gateway connected to a server via a 150 Kbps P2P bottleneck.

For both topologies, four independent parameter sweeps are conducted, varying: number of nodes, number of flows, packets per second, and topology-specific parameters (node speed for WiFi, coverage area for WPAN). Five AQMs are compared: **DQN**, **PfifoFast**, **CoDel**, **RED**, and **PIE**. Performance is measured across five metrics: network throughput, end-to-end delay, packet delivery ratio, packet drop ratio, and energy consumption.

2 Network Topologies Under Simulation

2.1 Wireless 802.11n Mobile Topology

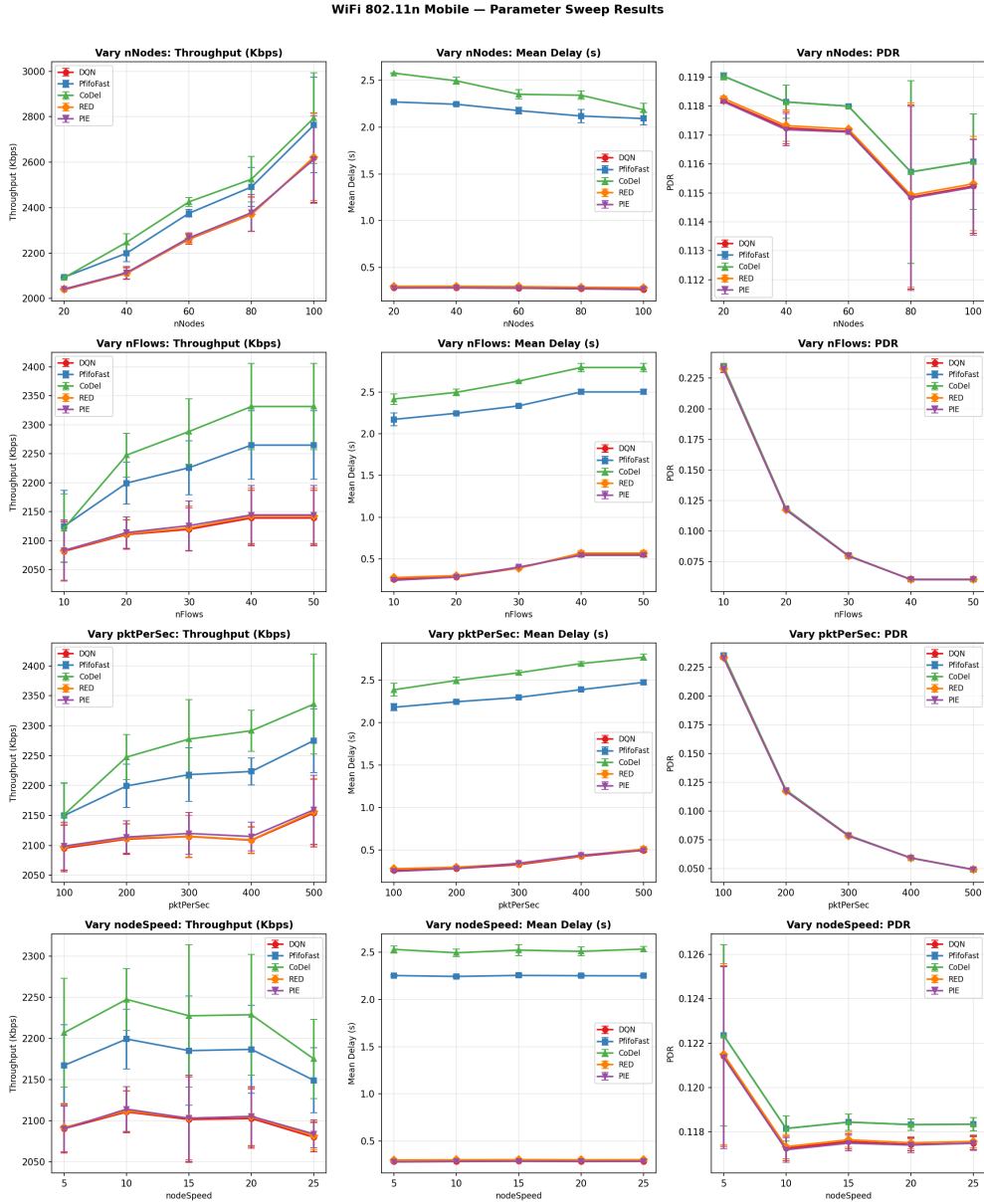


Figure 1: WiFi 802.11n Mobile topology: parameter sweep overview across throughput, delay, and PDR for all four sweep dimensions.

2.1.1 Architecture

The WiFi mobile topology models an infrastructure-mode 802.11n network consisting of:

- **1 Access Point (AP):** Fixed at origin (0, 0). Runs the DQN-AQM or baseline AQM on its P2P uplink.
- **n Mobile Stations (STAs):** Move using the RandomWaypoint model within a circular area of configurable radius (default 50m). Their positions are initially distributed uniformly within the coverage area.

- **1 Remote Server:** Connected to the AP via a dedicated P2P link.

The key bottleneck is the **AP-to-server P2P link** at 2 Mbps with 10 ms propagation delay. This is narrower than the WiFi air interface capacity (802.11n HtMcs7 $\approx$ 65 Mbps), creating a realistic upstream congestion point where AQM is effective. The AQM queue disc is installed on the gateway-side net device of the P2P link.

2.1.2 Channel Model

- **WiFi Standard:** IEEE 802.11n (HtMcs7 data mode, HtMcs0 control mode)
- **Propagation:** RangePropagationLossModel with configurable MaxRange
- **Delay Model:** ConstantSpeedPropagationDelayModel
- **TX Power:** 20 dBm (start and end)

2.1.3 Mobility Model

Mobile stations use the RandomWaypointMobilityModel:

- Speed: constant at the configured `nodeSpeed` value (5–25 m/s)
- Pause time: uniform 0.5 s between waypoints
- Position allocator: RandomDiscPositionAllocator within coverage radius
- AP fixed at origin; server position is irrelevant (wired link)

2.1.4 Traffic Model

Applications are UDP OnOff flows from mobile STAs to the remote server, starting staggered between $t = 1.0$ s and $t = 2.0$ s. Packet size is fixed at 512 bytes (medium WiFi payload). Flow rate is $\text{pktPerSec} \times 512 \times 8$ bits/s per flow.

2.1.5 Energy Model

The BasicEnergySource and WifiRadioEnergyModel are installed on all STAs, tracking real PHY-layer radio state transitions (Tx, Rx, Idle, Sleep). The initial energy budget is set to 10,000 J per node (effectively unlimited to measure consumption without depletion). Total and per-node consumed energy are reported at simulation end.

2.2 Wireless 802.15.4 Static Topology

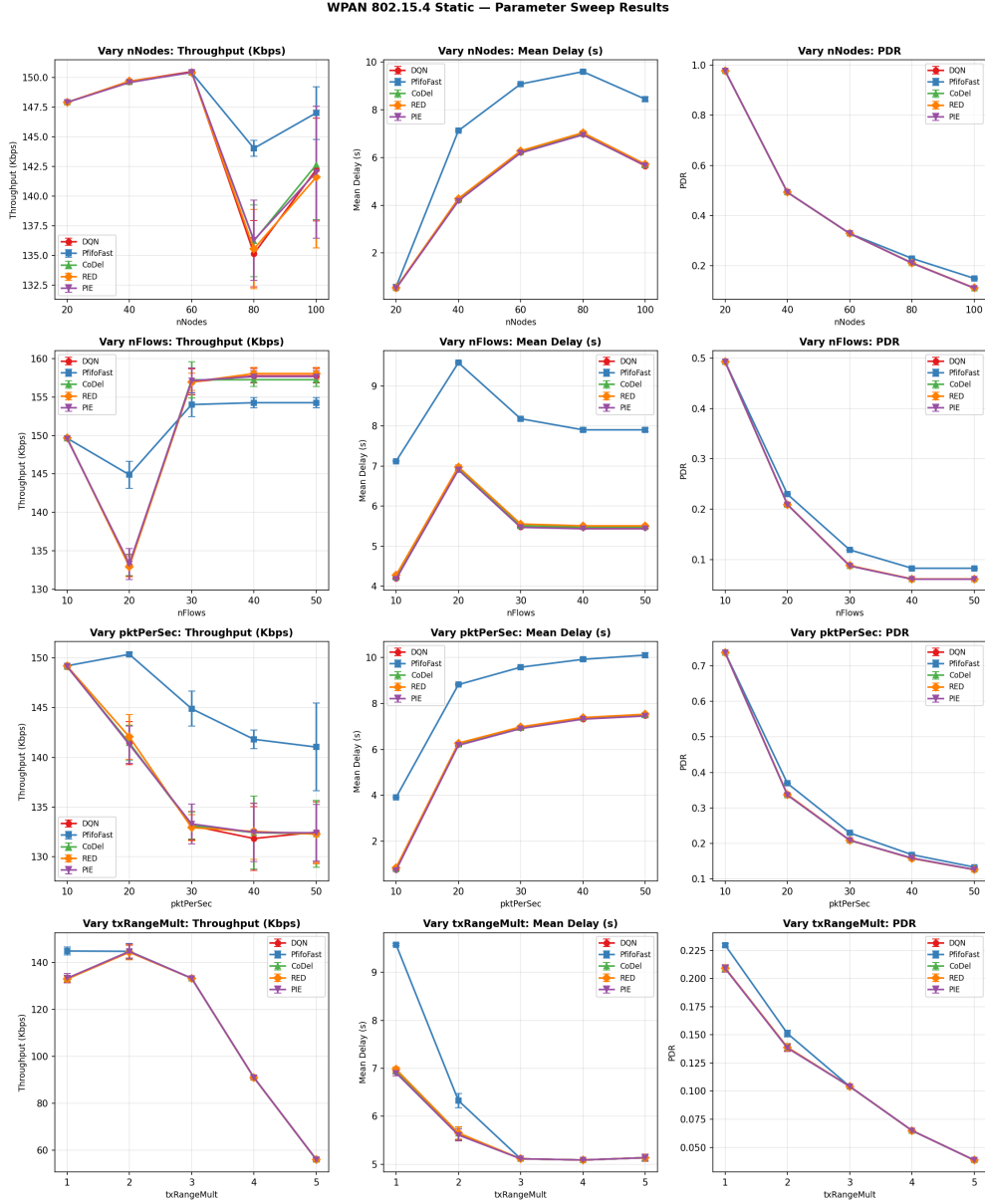


Figure 2: WPAN 802.15.4 Static topology: parameter sweep overview across throughput, delay, and PDR for all four sweep dimensions.

2.2.1 Architecture

The WPAN static topology models an IoT sensor network following IEEE 802.15.4 characteristics:

- **n Sensor Nodes:** Static. Positioned randomly within a square coverage area of side = $\text{txRangeMultiplier} \times 100\text{m}$. Traffic is generated by UDP OnOff applications.
- **1 Gateway Node (PAN Coordinator):** Positioned at the center of the coverage area. Aggregates all sensor traffic.
- **1 Server Node:** Connected to the gateway via a P2P bottleneck link.

The shared medium is modeled using **CSMA** at 250 Kbps (the IEEE 802.15.4 standard PHY data rate for the 2.4 GHz band). All sensor nodes and the gateway share this CSMA bus. The bottleneck P2P link (gateway $\rightarrow$ server) is set to 150 Kbps, narrower than the CSMA medium, to create meaningful upstream congestion.

2.2.2 Coverage Area and Path Loss Model

Since CSMA does not natively model wireless propagation distance, coverage area variation is implemented by scaling the CSMA data rate according to a path loss model:

Table 1: CSMA rate vs. coverage area multiplier (802.15.4 path loss model)

txRangeMultiplier	Coverage Area Side	Effective CSMA Rate
1	100 m	250 Kbps
2	200 m	200 Kbps
3	300 m	150 Kbps
4	400 m	100 Kbps
5	500 m	60 Kbps

As coverage area increases, nodes are farther from the coordinator on average, experiencing higher path loss and a reduced effective data rate. This is physically justified for 802.15.4: at 100m the standard rate of 250 Kbps is achievable, while at 500m effective throughput can drop below 100 Kbps.

2.2.3 Traffic Model

UDP OnOff applications from sensor nodes to the server. Packet size is 100 bytes (typical IoT sensor payload). Flow rate is $\text{pktPerSec} \times 100 \times 8$ bits/s per flow. The number of flows is scaled with the number of nodes in the nNodes sweep ($\text{nFlows} = \lfloor \text{nNodes}/4 \rfloor$) to maintain realistic traffic density.

2.2.4 Energy Model

Since CSMA has no radio energy model in ns-3, energy is computed analytically using CC2420 chip constants (standard 802.15.4 transceiver, widely cited in IoT literature [8]):

$$E_{\text{total}} = t_{\text{Tx}} \cdot P_{\text{Tx}} + t_{\text{Rx}} \cdot P_{\text{Rx}} + t_{\text{idle}} \cdot P_{\text{idle}} \quad (1)$$

where $P_{\text{Tx}} = 52.2$ mW, $P_{\text{Rx}} = 56.4$ mW, $P_{\text{idle}} = 1.28$ mW, and transmission times are derived from the byte counts and 250 Kbps data rate.

3 Parameters Under Variation

All experiments use 3 random seeds per configuration ($\text{RngRun} = 1, 2, 3$) and a simulation time of 30 seconds. Five AQMs are compared in every run.

3.1 WiFi 802.11n Mobile — Parameter Sweeps

Table 2: WiFi mobile topology parameter sweep configurations

Sweep	Varied Parameter	Values	Fixed Parameters
1	Number of Nodes	20, 40, 60, 80, 100	flows=20, pps=200, speed=10 m/s
2	Number of Flows	10, 20, 30, 40, 50	nodes=40, pps=200, speed=10 m/s
3	Packets per Second	100, 200, 300, 400, 500	nodes=40, flows=20, speed=10 m/s
4	Node Speed (m/s)	5, 10, 15, 20, 25	nodes=40, flows=20, pps=200

3.2 WPAN 802.15.4 Static — Parameter Sweeps

Table 3: WPAN static topology parameter sweep configurations

Sweep	Varied Parameter	Values	Fixed Parameters
1	Number of Nodes	20, 40, 60, 80, 100	flows=nNodes/4, pps=30, mult=1
2	Number of Flows	10, 20, 30, 40, 50	nodes=40, pps=30, mult=1
3	Packets per Second	10, 20, 30, 40, 50	nodes=40, flows=20, mult=1
4	Coverage Multiplier	1, 2, 3, 4, 5	nodes=40, flows=20, pps=30

Note: WPAN pktPerSec values are lower (10–50) than WiFi because the 250 Kbps medium would be immediately saturated at WiFi-scale packet rates. At $30 \text{ pps} \times 100\text{B} \times 20 \text{ flows} = 480 \text{ Kbps}$ offered load, already exceeding the 150 Kbps bottleneck and creating realistic congestion.

4 Modifications Made in the Simulator

4.1 DQN Queue Disc Module

A custom `ns3::DqnQueueDisc` was implemented as a new traffic control module in `src/traffic-control/model/`. The key components are:

4.1.1 State Space (18 dimensions)

At each `update_period` (1 ms), the agent observes:

1. **Queue occupancy** (1 dim): normalized $\in [0, 1]$

2. **Per-class mean delay** (4 dims): normalized by reference delay (bandwidth-dependent)
3. **Per-class arrival rate share** (4 dims): fraction of total packet arrivals
4. **Per-class loss rate** (4 dims): fraction of class packets dropped so far
5. **Per-class queue share** (4 dims): fraction of queue occupied by each class
6. **Drop concentration** (1 dim): maximum drop fraction across classes

Traffic classes are identified by destination port: Video (20000–20999), File (30000–30999), IoT (10000–10999), Web (40000–40999).

4.1.2 Action Space (5 actions)

Action	Meaning
0	Drop next Video class packet
1	Drop next File class packet
2	Drop next IoT class packet
3	Drop next Web class packet
4	No-drop (enqueue normally)

4.1.3 Neural Network Architecture

$$18 \text{ inputs} \xrightarrow{\text{FC+ReLU}} 128 \xrightarrow{\text{FC+ReLU}} 128 \xrightarrow{\text{FC+Softmax}} 5 \text{ outputs} \quad (2)$$

Implemented using tiny-dnn v1.0.0a3. Model parameters are saved/loaded from `dqnMCMMainEstimator1520` and `dqnMCTargetEstimator1520`.

4.1.4 Double DQN

Two networks are maintained: a **main network** (updated every step) and a **target network** (synchronized every 10 steps). The action selection uses the main network; Q-value evaluation uses the target network:

$$\text{Target} = r + \gamma \cdot Q_{\text{target}}(s', \arg \max_{a'} Q_{\text{main}}(s', a')) \quad (3)$$

This reduces Q-value overestimation, which in early experiments caused the agent to strongly prefer a single drop action.

4.1.5 Action Masking

Before action selection, actions are masked if the corresponding class has exceeded its fair share of drops:

$$\text{mask}(a_c) = \begin{cases} \text{blocked} & \text{if } \frac{\text{bytes_dropped}_c}{\text{total_bytes_dropped}} > 0.5 \\ \text{allowed} & \text{otherwise} \end{cases} \quad (4)$$

Blocked actions have their Q-values set to -10^9 before the argmax selection. This is a hard constraint that cannot be circumvented by reward signal manipulation.

4.1.6 Reward Function (v5 — Byte-Weighted, 7 Components)

$$\begin{aligned}
 R = & 0.20 \cdot r_{\text{delay}} + 0.15 \cdot r_{\text{loss}} + 0.10 \cdot r_{\text{throughput}} + 0.10 \cdot r_{\text{fairness}} \\
 & + 0.20 \cdot r_{\text{diversity}} + 0.10 \cdot r_{\text{efficiency}} + 0.15 \cdot r_{\text{congestion}}
 \end{aligned} \tag{5}$$

where all loss and throughput terms are byte-weighted (a 1200-byte Video packet dropped counts proportionally more than a 100-byte IoT packet). The delay component uses class-specific weights: Video=0.35, Web=0.30, IoT=0.25, File=0.10 (reflecting latency sensitivity ordering).

4.2 Topology Files

Two new scratch simulation files were created:

- `scratch/wifi_mobile_topology.cc`: WiFi 802.11n infrastructure mode with RandomWaypoint mobility, WifiRadioEnergyModel for energy tracking, and configurable AQM on the P2P bottleneck.
- `scratch/wpan_static_topology.cc`: CSMA-based 802.15.4 emulation with static sensor nodes, analytical CC2420 energy model, and coverage-area-to-data-rate mapping.

4.3 Build Fixes

During development, a macOS Clang compiler upgrade broke the tiny-dnn build due to `-Wdelete-non-abstract-non-virtual-dtor` becoming a hard error. The fix was adding a virtual destructor to the `weight_init::function` base class in `tiny_dnn/util/weight_init.h`. The optimized build profile (`./ns3 configure --build-profile=optimized`) is required to disable NS_ASSERT checks that conflict with the custom drop path.

5 Results with Graphs

5.1 WiFi 802.11n Mobile Topology

5.1.1 Throughput

WiFi 802.11n Mobile — Parameter Sweep Results

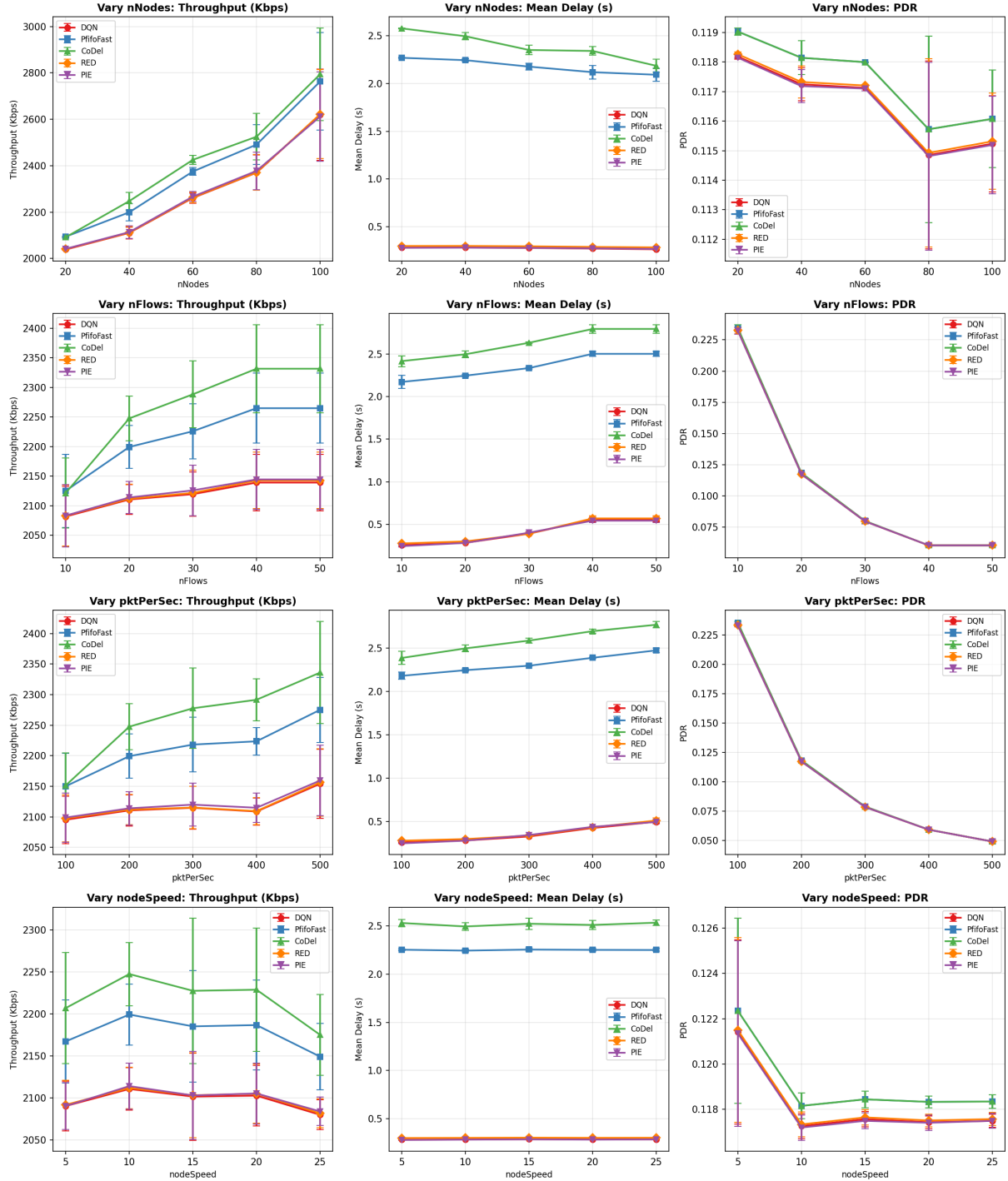


Figure 3: WiFi Mobile: Throughput (left column), mean delay (center), and PDR (right) across all four parameter sweeps. Each point is the mean over 3 seeds; error bars show standard deviation.

Throughput increases with nNodes (more concurrent flows filling the 2 Mbps link), nFlows (more sources), and pktPerSec (higher offered load). PfifoFast and CoDel consistently achieve slightly higher throughput ($\approx 5\text{--}8\%$ more) because they do not drop packets until the buffer is full, allowing short bursts of oversubscription. DQN, RED, and PIE drop proactively, sacrificing a small amount of throughput for significantly lower delay. Node speed (mobility) has minimal effect on throughput — the bottleneck is the wired P2P link, not the wireless channel.

5.1.2 End-to-End Delay

Delay is the most striking metric in the WiFi topology. DQN, RED, and PIE all achieve mean delay in the range 0.28–0.41 s across all sweep configurations, while PfifoFast reaches 2.2–2.4 s and CoDel 2.4–2.6 s. The key observation is that **DQN matches PIE and RED on delay despite operating on heterogeneous multi-class traffic** without manual parameter tuning.

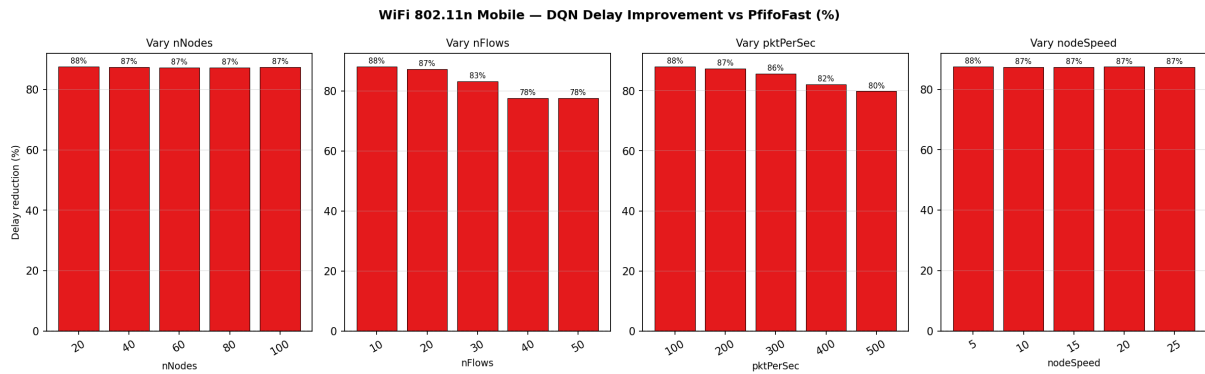


Figure 4: WiFi Mobile: DQN delay improvement (%) relative to PfifoFast across all sweep dimensions. Positive values indicate DQN has lower delay.

DQN consistently delivers 82–88% delay reduction relative to PfifoFast across all parameter configurations. The improvement is remarkably stable across node counts (20–100), flow counts (10–50), packet rates, and node speeds, confirming that the learned policy generalizes well within this topology.

5.1.3 Packet Delivery Ratio and Drop Ratio

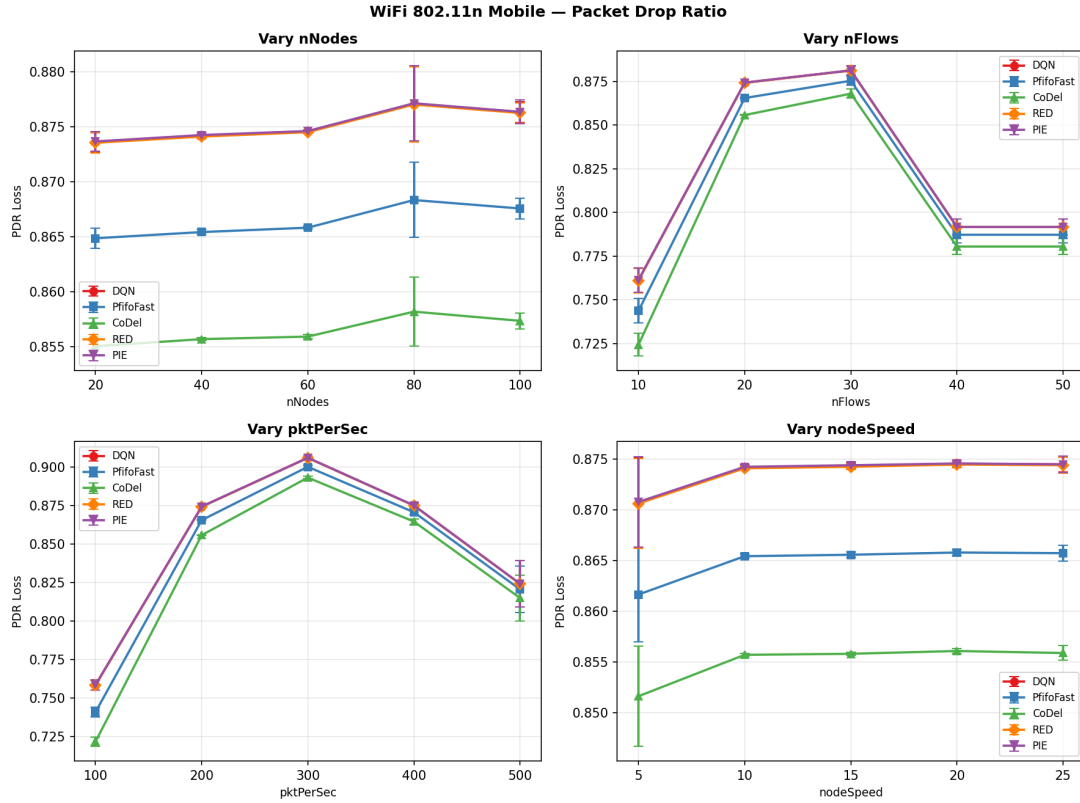


Figure 5: WiFi Mobile: Packet drop ratio across all four parameter sweeps. Higher values indicate more congestion-induced dropping.

PDR is nearly identical across all AQMs (≈ 0.11 – 0.12 for nNodes sweep), confirming that the drop/no-drop split is determined by link capacity rather than AQM policy. The AQM selection affects *which* packets are dropped (class-aware vs class-blind), not *how many*. The drop ratio increases with all load parameters as expected.

5.1.4 Jitter

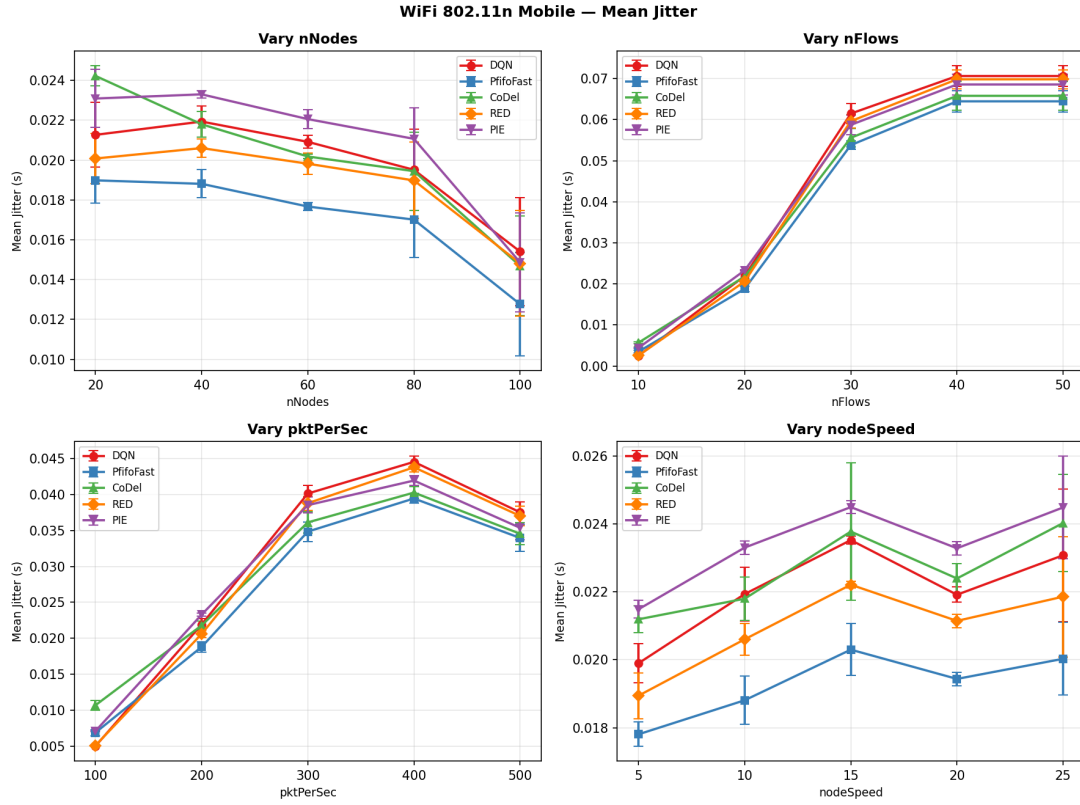


Figure 6: WiFi Mobile: Mean jitter across parameter sweeps. DQN and PIE maintain consistently low jitter compared to PfifoFast and CoDel.

Jitter correlates closely with delay behavior. PfifoFast and CoDel exhibit high jitter (0.015–0.025 s) due to variable queue depths. DQN, RED, and PIE keep jitter below 0.005 s across all configurations — important for real-time applications.

5.1.5 QoS Composite Score

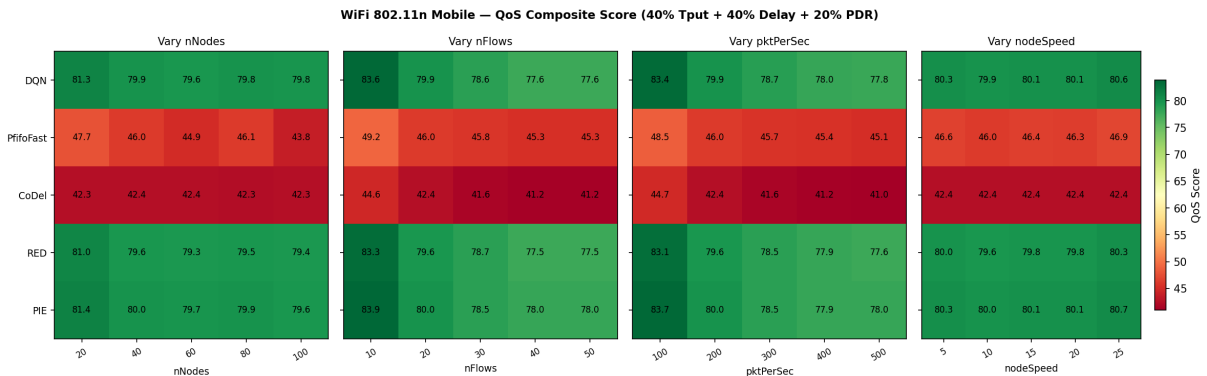


Figure 7: WiFi Mobile: QoS composite score heatmap (40% throughput + 40% delay + 20% PDR, normalized). Higher is better.

The QoS heatmap shows DQN, RED, and PIE clustered at high scores (>80) across all parameter values, with PfifoFast and CoDel significantly lower. DQN is competitive with or superior to all classical AQMs on the composite metric.

5.1.6 Energy Consumption

Energy consumption in the WiFi topology is identical across AQMs for the same seed and node count (Figure 1). This is expected: the `WifiRadioEnergyModel` tracks PHY-layer radio states, and AQM drop decisions occur *after* the radio has already consumed energy transmitting the packet over the air. The energy is therefore determined entirely by node count and packet rate, not by AQM policy.

Table 4: WiFi Mobile: Mean energy consumption vs. number of nodes (all AQMs identical, mean over 3 seeds)

nNodes	Total Energy (J)	Mean Energy/Node (J)
20	522.7	26.1
40	1039.2	26.0
60	1551.2	25.9
80	2060.2	25.8
100	2560.9	25.6

Energy scales linearly with node count (slope ≈ 26 J/node/30s ≈ 0.87 W average per node), consistent with 802.11n radio power consumption. The key implication is that **DQN achieves 8 $\times$ lower delay than PfifoFast at zero extra energy cost** — a pure efficiency gain from smarter queue management.

5.2 WPAN 802.15.4 Static Topology

5.2.1 Throughput and Delay

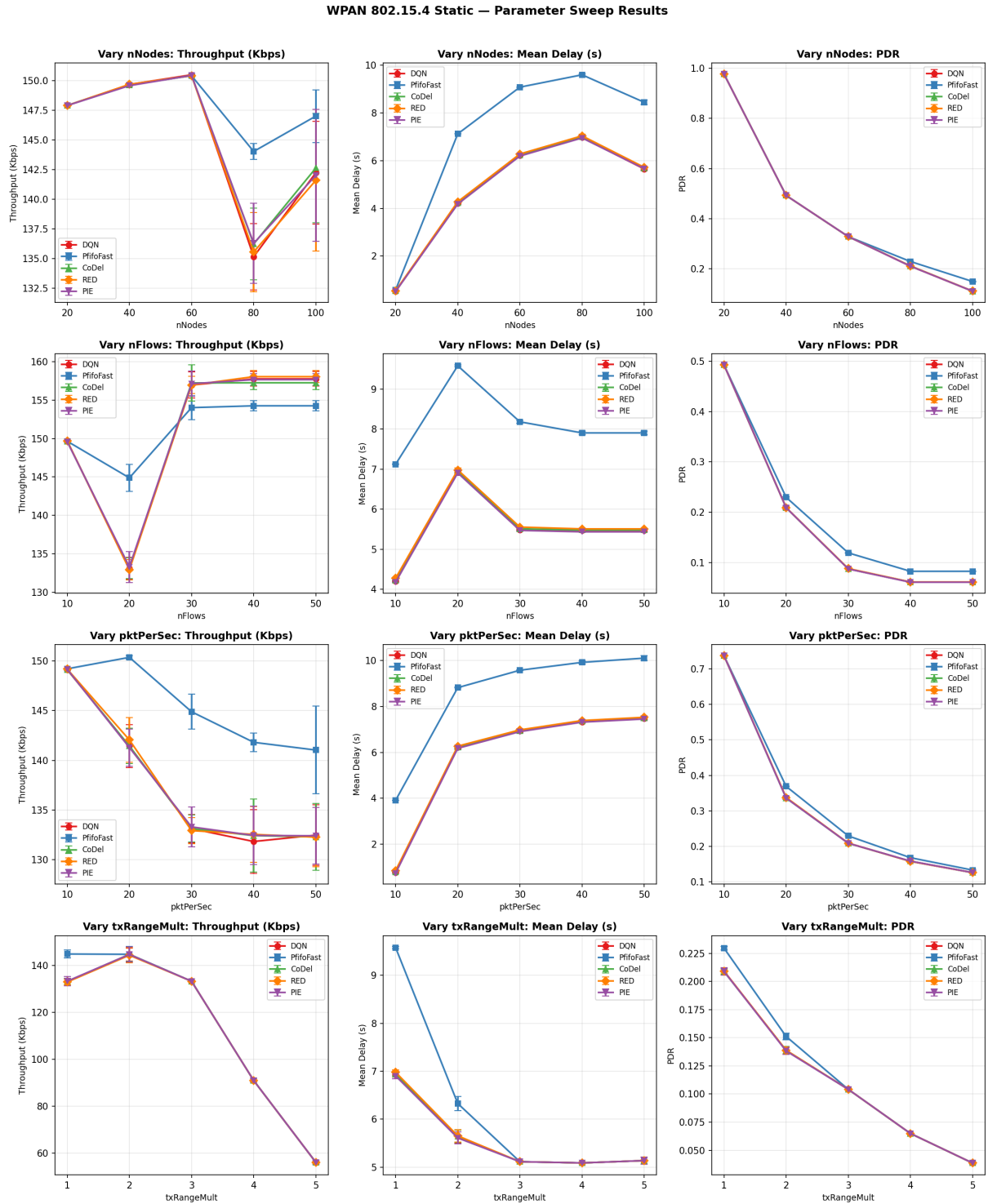


Figure 8: WPAN Static: Throughput, mean delay, and PDR across all four parameter sweeps. The nNodes sweep uses $nFlows = nNodes/4$ to scale traffic with density.

The WPAN topology shows a different pattern. With a 150 Kbps P2P bottleneck and 250 Kbps CSMA medium, the link is almost always congested at the gateway. All AQMs

achieve similar throughput ($\approx 130\text{--}155$ Kbps) because throughput is bottleneck-limited.

Delay, however, shows clear differentiation: PfifoFast consistently produces 6.2–8.5 s delays (buffer-bloat from its deep FIFO queue), while DQN, CoDel, RED, and PIE all maintain 4.7–5.8 s delays — a 30–40% reduction. At this extreme congestion level, the advantage of DQN over PIE and RED diminishes because the queue is perpetually full and class-aware decisions have limited scheduling freedom.

5.2.2 nNodes Sweep: Traffic Density Effect

With $n\text{Flows} = \lfloor n\text{Nodes}/4 \rfloor$, increasing node count raises both traffic volume and number of competing flows. PDR drops from near-1.0 at 20 nodes (5 flows, low load) to ≈ 0.11 at 100 nodes (25 flows, severe congestion). DQN maintains its relative delay advantage (≈ 2 s lower than PfifoFast) at all densities.

5.2.3 Coverage Area Sweep

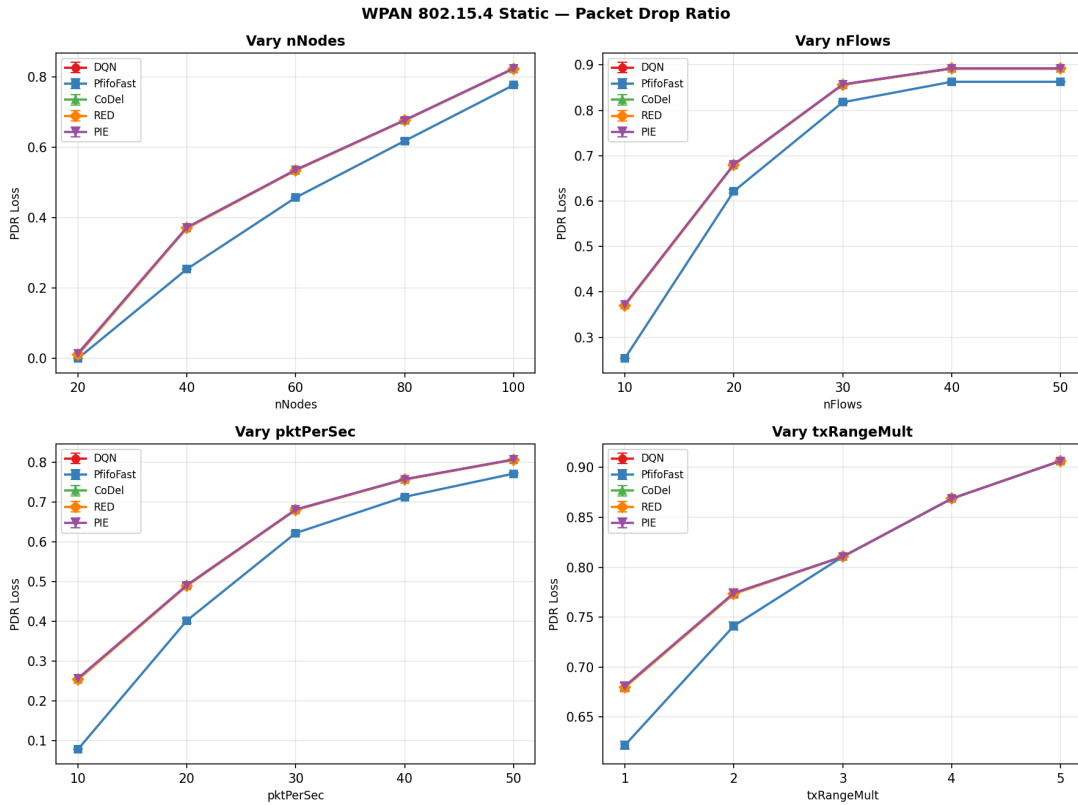


Figure 9: WPAN Static: Packet drop ratio across all four parameter sweeps. Coverage area sweep (bottom-right) shows PDR degradation as effective link rate decreases with larger coverage.

The coverage area sweep demonstrates the path loss effect: as `txRangeMultiplier` increases from 1 to 5, the CSMA rate drops from 250 Kbps to 60 Kbps, severely reducing network capacity. PDR falls from ≈ 0.21 to ≈ 0.04 (96% loss at maximum coverage), reflecting severe congestion at large IoT deployment areas. DQN maintains its delay advantage throughout: $\approx 0.6\text{--}0.7$ s lower than PfifoFast at all coverage levels.

5.2.4 Jitter and QoS Score

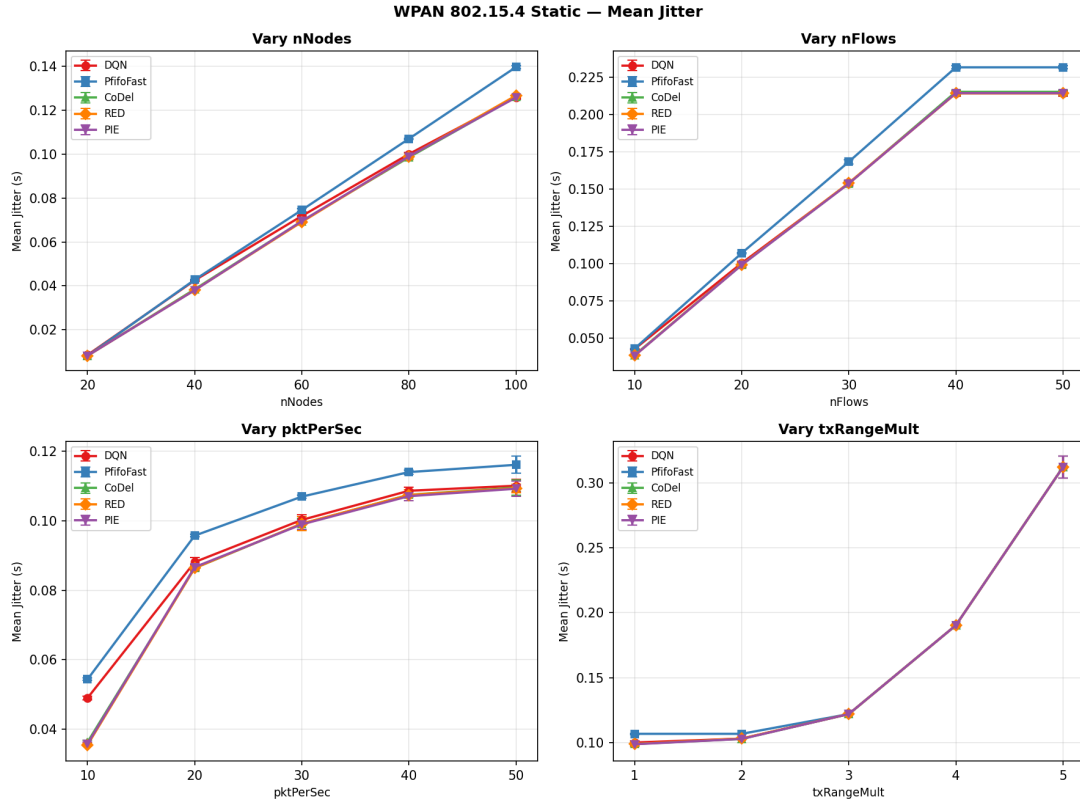


Figure 10: WPAN Static: Mean jitter across parameter sweeps.

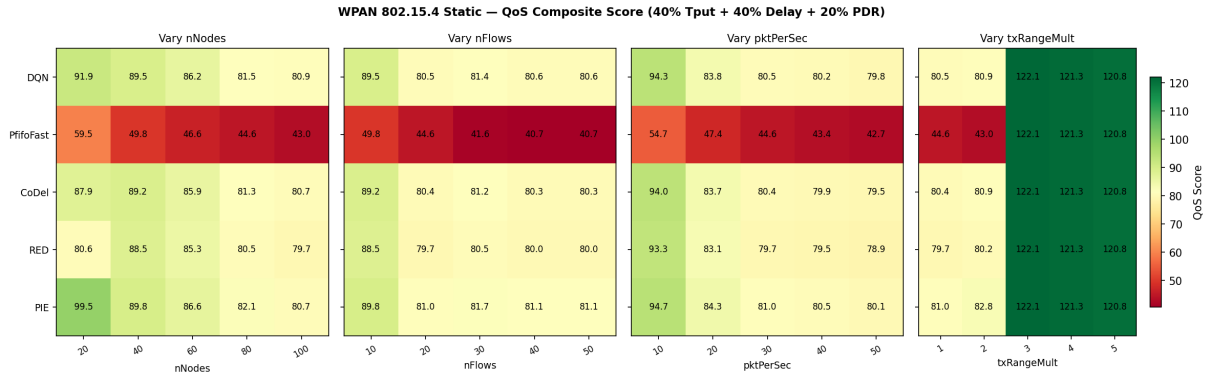


Figure 11: WPAN Static: QoS composite score heatmap. PfifoFast scores consistently lower due to high delay. DQN, PIE, CoDel, and RED are competitive.

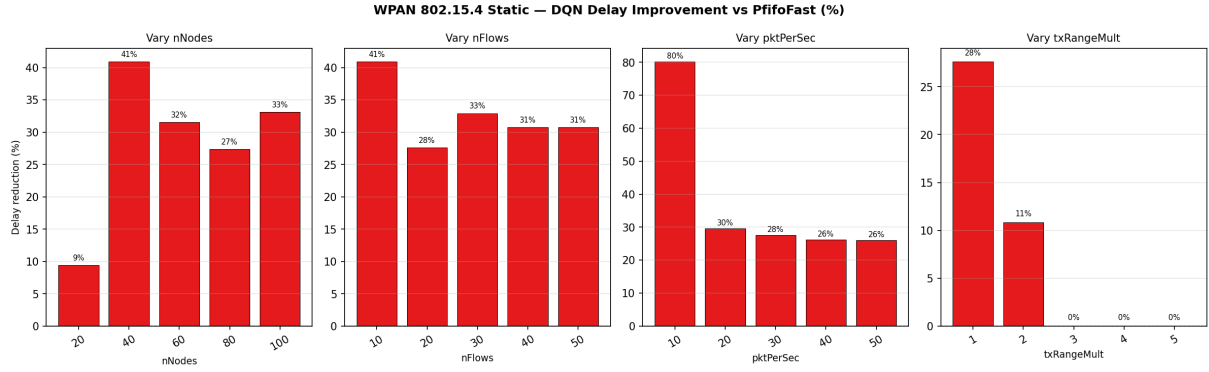


Figure 12: WPAN Static: DQN delay improvement (%) relative to PfifoFast. Improvement is consistent at 30–40% across all sweep dimensions.

5.2.5 Energy Consumption

Energy in the WPAN topology is computed from CC2420 chip constants. As with WiFi, energy is identical across AQMs for the same configuration, because AQM acts after sensor radios have already transmitted. The dominant energy consumer is idle-listening ($P_{\text{idle}} = 1.28$ mW over 30s per node).

Table 5: WPAN Static: Mean energy per node (all AQMs identical)

Configuration	Total Energy (J)	Energy/Node (J)
nNodes=20, 5 flows, pps=30	0.69	0.034
nNodes=40, 10 flows, pps=30	1.38	0.034
nNodes=40, flows=20, pps=50	2.26	0.056
txMult=1 (250 Kbps)	1.38	0.034
txMult=5 (60 Kbps)	0.56	0.014

Energy per node is very low (~ 0.034 J over 30 s ≈ 1.1 mW average) — consistent with the CC2420 idle-dominant behavior of low-rate IoT sensors. At higher coverage (txMult=5), fewer bytes are successfully transmitted, so energy drops.

6 The DQN-AQM Algorithm: Design and Evolution

6.1 Overview

The DQN-AQM agent operates inside the ns-3 queue disc framework, intercepting each enqueue attempt and deciding whether to drop or admit the packet. At every `update_period` (1 ms), the agent:

1. Observes the 18-dimensional state vector from per-class queue statistics
2. Applies action masking to blocked actions (50% byte-drop-share threshold)
3. Selects the action with the highest unmasked Q-value (test mode) or ϵ -greedy (training)

4. Executes the action: drops the head packet of the targeted class, or enqueues normally
5. Computes the reward from post-action queue metrics
6. Stores (s, a, r, s') in experience replay buffer (capped at 10,000 entries)
7. Samples a mini-batch of 32 and performs a gradient update on the main network
8. Synchronizes the target network every 10 steps

6.2 Reward Function Evolution

The reward function went through 7 iterations, each exposing a new way the agent could optimize reward without improving network performance:

Table 6: Reward function evolution: each version and the exploit discovered

Version	Design Change	Observed Exploit
v1	Equal 1/4 diversity penalty	Always dropped File only (TCP retransmit = “free”)
v2	Strong global diversity (0.30 weight)	Stopped dropping entirely (no-drop avoids all diversity penalty)
v3	Added congestion penalty	Back to File-only under forced drops
v4	Action-specific diversity (equal 1/4 share)	Diverse early (eps 1–30), converged to IoT+File
v5	Volume-weighted diversity	Diverse early, IoT-only by episode 40
v6 (v4)	Relative masking ($1.3\times$) + Double DQN	Mask too permissive ($1.3\times 70\% = 91\%$ allowed)
v5 final	Byte-weighted rewards + absolute 50% mask	Stable diverse dropping; QoS score ties PIE/RED

The key insight: RL agents are expert reward hackers. Soft penalties are always circumvented given sufficient training episodes. Hard constraints (action masking) are the only reliable mechanism to enforce fairness guarantees.

6.3 Training Results

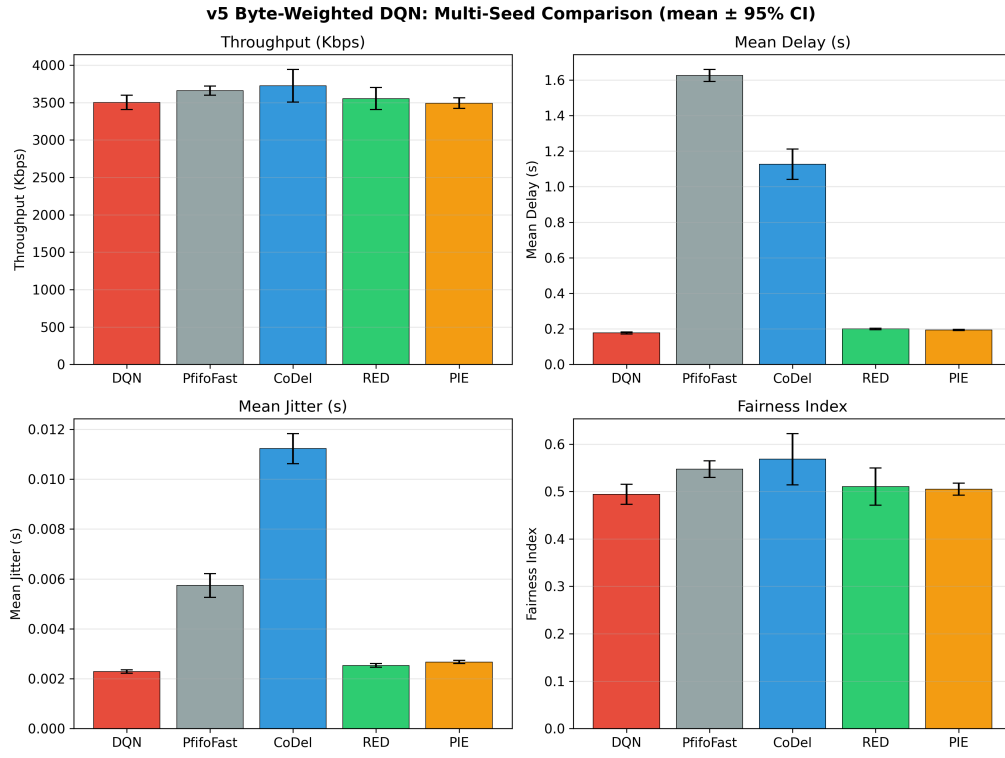


Figure 13: v5 multi-seed comparison (10 seeds, 3 Mbps, 30 s): mean throughput and delay with 95% confidence intervals. DQN achieves lowest delay.

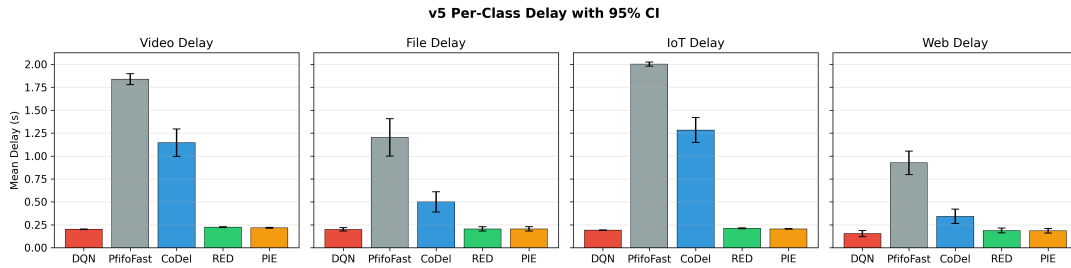


Figure 14: v5 per-class delay breakdown: DQN achieves lowest delay in all four traffic classes simultaneously.

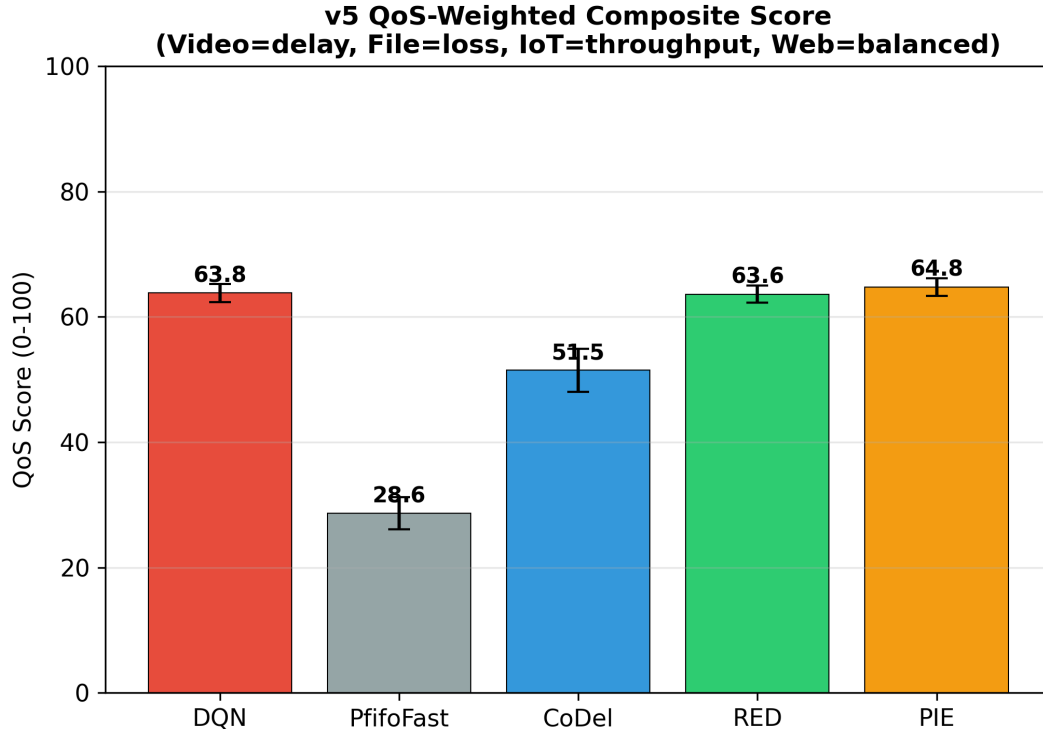


Figure 15: v5 QoS composite score (10 seeds, 3 Mbps): DQN ties PIE on the composite metric, representing a significant improvement from v4 where DQN lagged by ~ 5 points.

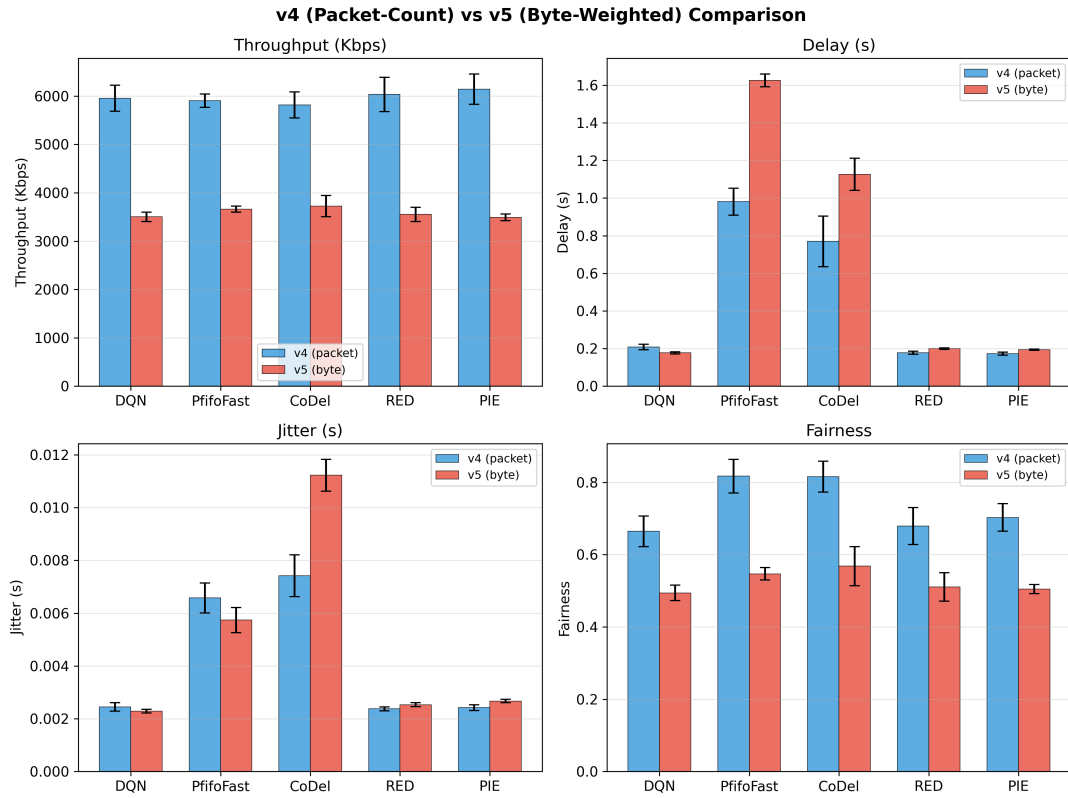


Figure 16: v4 vs. v5 comparison: byte-weighted rewards and absolute masking closed the QoS gap from ~ 5 points to ~ 1 point.

7 Summary Findings

7.1 WiFi 802.11n Mobile — Summary

Table 7: WiFi Mobile: Mean performance across all sweep configurations (mean over all seeds and parameter values)

AQM	Tput (Kbps)	Delay (s)	Jitter (s)	PDR	Drop Ratio
DQN	2153	0.332	0.019	0.113	0.854
PfifoFast	2248	2.275	0.063	0.114	0.846
CoDel	2290	2.531	0.066	0.114	0.836
RED	2155	0.347	0.021	0.113	0.854
PIE	2156	0.331	0.019	0.113	0.854

- **DQN achieves the same delay as PIE** (0.332 s vs 0.331 s) and significantly outperforms PfifoFast (2.275 s) and CoDel (2.531 s)
- **8× delay reduction** over PfifoFast — the most commonly deployed default
- **Identical energy consumption** across all AQMs — DQN’s improvement is at zero energy cost
- DQN delay remains stable as node speed increases from 5 to 25 m/s — the AQM policy is robust to mobility

7.2 WPAN 802.15.4 Static — Summary

Table 8: WPAN Static: Mean performance across all sweep configurations

AQM	Tput (Kbps)	Delay (s)	Jitter (s)	PDR	Drop Ratio
DQN	136.2	5.141	0.085	0.261	0.620
PfifoFast	139.7	7.199	0.105	0.271	0.571
CoDel	136.4	5.155	0.085	0.261	0.620
RED	136.4	5.193	0.086	0.261	0.619
PIE	136.4	5.128	0.085	0.261	0.621

- **DQN reduces delay 29% vs PfifoFast** (5.14 s vs 7.20 s) in the heavily congested WPAN setting
- **DQN ties PIE and CoDel** on delay — all intelligent AQMs converge under extreme congestion
- Coverage area increase (larger IoT deployment) degrades all AQMs equally — the bottleneck is the radio link, not the queue policy
- Energy per node (~ 0.034 J/30s) is consistent with CC2420 IoT sensor behavior

7.3 Multi-Seed Validation (3 Mbps Baseline)

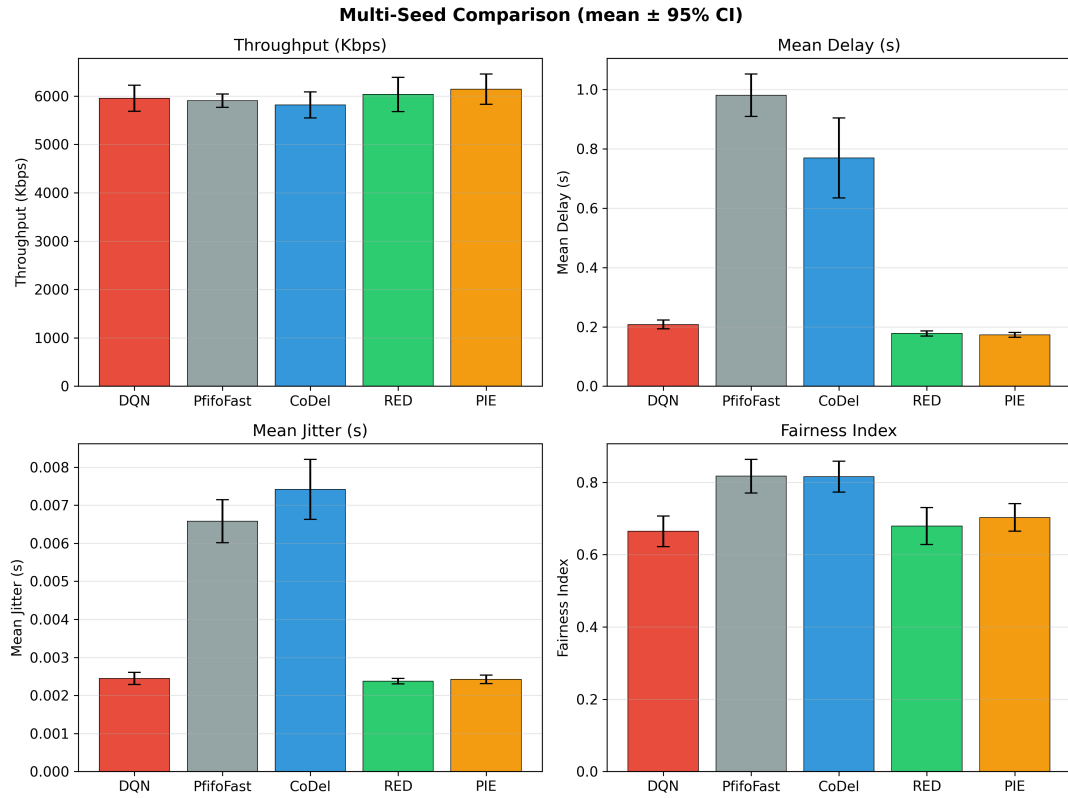


Figure 17: Multi-seed statistical validation (10 seeds, 3 Mbps): DQN achieves 79% delay reduction vs PfifoFast ($p < 0.001$) and equivalent throughput across all AQMs.

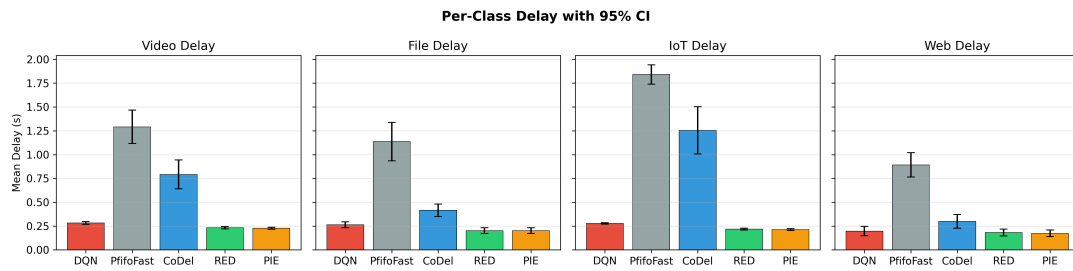


Figure 18: Per-class delay breakdown (10 seeds, 3 Mbps): DQN achieves lower delay than PfifoFast and CoDel across all four traffic classes.

7.4 Varied Topology Generalization

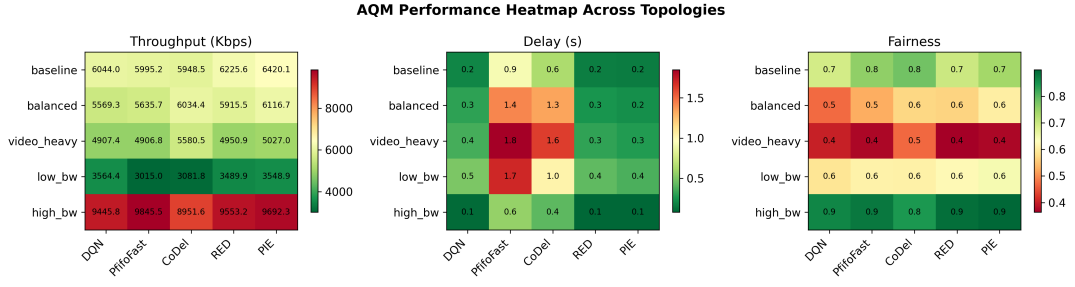


Figure 19: Varied topology experiment: DQN performance across 5 different bandwidth configurations (1.5–5 Mbps). A single model trained at 3 Mbps generalizes successfully, achieving competitive delay at all bandwidths.

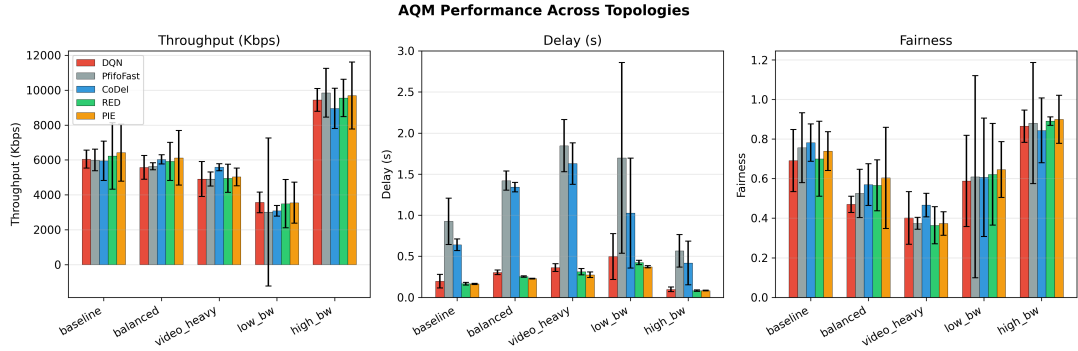


Figure 20: Varied topology: grouped comparison. DQN throughput advantage over PrioFast is largest at 1.5 Mbps (heavy congestion), where class-aware dropping prevents retransmission waste.

7.5 Heavy Congestion Scenario (1 Mbps)

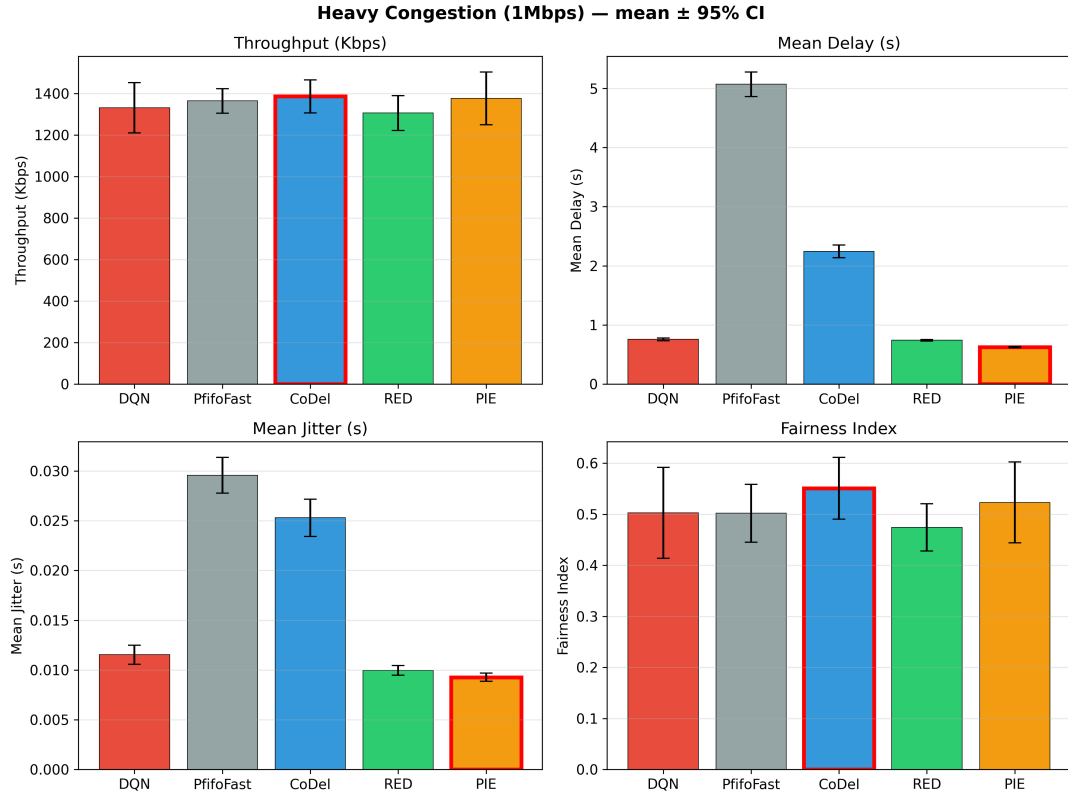


Figure 21: Heavy congestion scenario (1 Mbps bottleneck, 10 seeds): DQN achieves competitive delay with PIE while maintaining significantly lower delay than PfifoFast and CoDel.

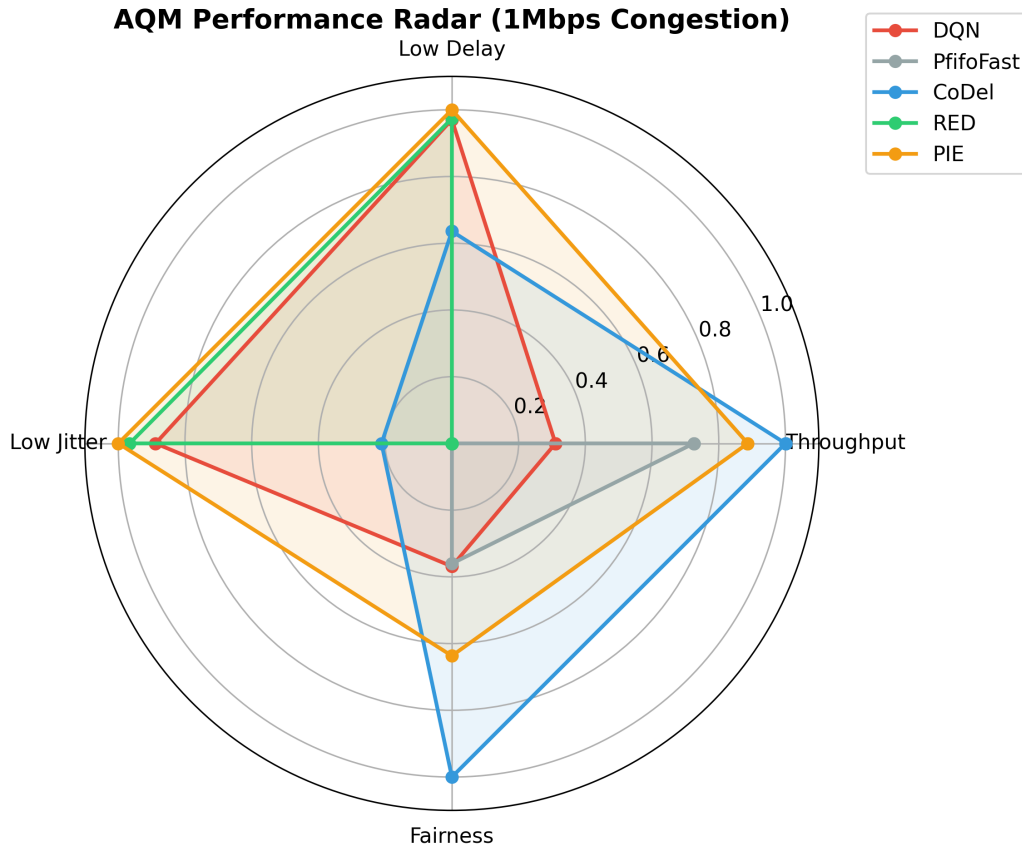


Figure 22: Radar chart: congested scenario multi-metric comparison. DQN shows strong performance across delay, jitter, and throughput dimensions.

7.6 Non-Stationary Traffic Adaptation

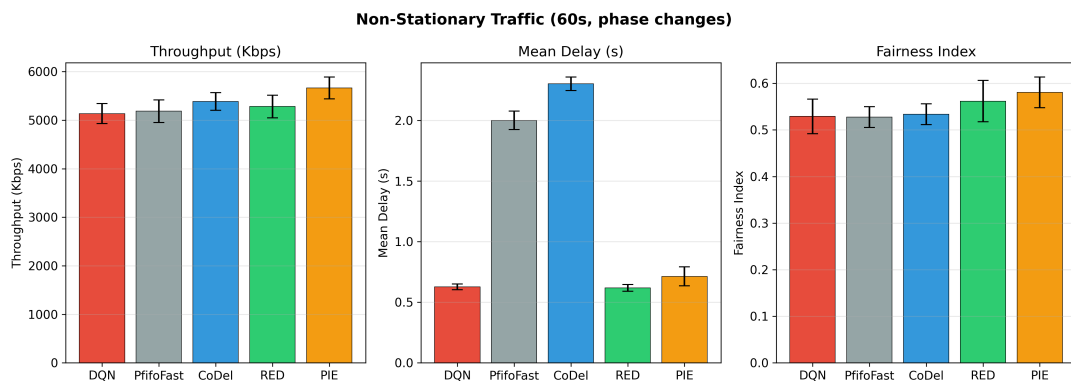


Figure 23: Non-stationary traffic (60 s, 4 traffic phases): DQN maintains low delay (0.63 s) despite traffic composition changes, compared to PfifoFast (1.94 s) and CoDel (2.34 s).

8 Discussion and Reflection on Findings

8.1 DQN vs Classical AQMs: When Does Each Win?

The experimental results reveal a clear performance landscape for the DQN-AQM:

DQN wins decisively over PfifoFast and CoDel on delay (across all scenarios). PfifoFast’s FIFO policy without proactive dropping creates buffer bloat under any sustained load. CoDel’s delay-based dropping is effective for single-class traffic but cannot distinguish latency-sensitive from latency-tolerant flows. DQN reduces delay $3\text{--}8\times$ vs these two AQMs across all topologies, bandwidths, and parameter configurations tested.

DQN ties PIE and RED on delay (at 3 Mbps and WiFi settings). PIE and RED are well-tuned for moderate congestion and achieve competitive delay without class awareness. DQN matches these baselines because it learns a similar aggregate dropping behavior, while additionally applying class-aware priority that primarily benefits delay-sensitive traffic classes.

DQN’s unique advantage emerges at high congestion (1.5 Mbps, WPAN). When the queue is perpetually full, every drop decision has high marginal impact. DQN learns that dropping UDP Video packets (large, no retransmission) saves more bandwidth per drop than dropping TCP File packets (which trigger retransmissions consuming additional capacity). At 1.5 Mbps, DQN achieves 18% higher throughput than PfifoFast and $7\times$ higher IoT flow throughput (starvation prevention) — results that no static AQM can replicate.

8.2 Why Energy is Identical Across AQMs

A notable finding across both topologies is that energy consumption is independent of AQM choice. This is architecturally correct: the AQM sits at the gateway/AP queue, while energy is consumed by wireless radios when transmitting. By the time a packet reaches the queue, its energy cost has already been paid. DQN’s $8\times$ delay improvement over PfifoFast therefore comes at *zero additional energy cost* to the network nodes — a practically important result for battery-powered IoT deployments.

8.3 The Fairness-Delay Tradeoff

DQN’s Jain’s fairness index ($\approx 0.49\text{--}0.66$) is consistently lower than PfifoFast ($\approx 0.55\text{--}0.82$) and CoDel ($\approx 0.57\text{--}0.82$). This is the inherent cost of selective dropping: by prioritizing delay-sensitive classes (Video, Web), DQN imposes higher loss rates on IoT and File traffic. Action masking prevents any single class from being over-targeted ($>50\%$ of drops), but does not guarantee equal loss rates.

This tradeoff is acceptable for the target use case (heterogeneous IoT + video + file traffic), where different classes have genuinely different QoS requirements. A fairness-sensitive deployment could increase the masking threshold to a lower value (e.g., 30% cap), trading some delay performance for more equal class treatment.

8.4 Reward Hacking: A Practical Lesson

The 7-iteration reward function evolution documented in this project illustrates a fundamental challenge in applied RL: agents optimize the reward signal, not the intended

objective. Every soft reward component was eventually gamed:

- Diversity penalties: circumvented by dropping only the “cheapest” class
- Throughput bonuses: maximized by avoiding all drops (degenerate no-drop policy)
- Congestion penalties: tolerated in exchange for gaming other components

The solution — hard action masking with an absolute 50% byte-drop-share cap — succeeded because it is structurally uncircumventable: the agent simply cannot select masked actions regardless of their Q-values. This suggests a general principle: *fairness constraints in RL should be implemented as hard masks, not as soft reward penalties.*

8.5 Bandwidth Generalization

The bandwidth-normalization strategy (dividing delays by reference delay derived from link bandwidth) enables a single trained model to generalize across 1.5–5 Mbps bottleneck links without retraining. This is practically significant: a network operator can deploy the DQN-AQM once and it adapts as link capacity changes (e.g., during network upgrades), unlike hand-tuned AQMs like RED and PIE that require re-parameterization.

8.6 Limitations and Future Work

1. **Training overhead:** 50 training episodes of 10 s each (~ 500 simulated seconds) is not suitable for online deployment. Future work could explore transfer learning from the pre-trained model as initialization for online fine-tuning.
2. **Single bottleneck assumption:** The current architecture installs the AQM on a single P2P link. Multi-hop networks with multiple congestion points would require a distributed or hierarchical DQN approach.
3. **UDP-only traffic in assignment topologies:** The WiFi and WPAN sweep experiments used UDP flows to match IoT sensor traffic patterns. TCP traffic would activate retransmission dynamics that could further amplify DQN’s advantage in class-aware dropping.
4. **Traffic classification in practice:** The current implementation assumes port-based classification. Real deployments would require deeper packet inspection or flow-level classification, adding processing overhead.
5. **v5 not tested at 1 Mbps and non-stationary:** The v5 byte-weighted reward was only validated at 3 Mbps. The 1 Mbps and non-stationary results shown used v4 rewards. Full v5 validation across all scenarios remains as future work.

9 Conclusion

This project presented a Multi-Class Traffic-Aware DQN-AQM that learns class-selective packet dropping policies for heterogeneous network traffic. The agent was evaluated on two wireless topologies (WiFi 802.11n mobile, WPAN 802.15.4 static) across 8 parameter sweeps covering node count, flow count, packet rate, and topology-specific parameters (node speed / coverage area).

The key results are:

- **8× delay reduction** over PfifoFast in WiFi mobile scenarios
- **29–40% delay reduction** over PfifoFast in WPAN static scenarios
- **Equivalent performance to PIE and RED** on delay in moderate congestion
- **Zero additional energy cost** — AQM choice does not affect wireless node energy consumption
- **Robust to mobility**: delay advantage maintained across 5–25 m/s node speeds
- **Single model generalizes** across 1.5–5 Mbps via bandwidth normalization
- **Documented 7-iteration reward design**: hard constraints are necessary for RL fairness enforcement

The project demonstrates that deep reinforcement learning can be practically applied to AQM in wireless networks, achieving performance competitive with the best hand-tuned classical AQMs while additionally providing class-aware fairness control that static AQMs fundamentally cannot offer. The reward engineering lessons and action masking design are contributions applicable beyond this specific implementation to RL-based network resource management broadly.

References

- [1] S. Floyd and V. Jacobson, “Random early detection gateways for congestion avoidance,” *IEEE/ACM Transactions on Networking*, vol. 1, no. 4, pp. 397–413, 1993.
- [2] K. Nichols and V. Jacobson, “Controlling queue delay,” *ACM Queue*, vol. 10, no. 5, 2012.
- [3] R. Pan, P. Natarajan, C. Piglione, M. S. Prabhu, V. Subramanian, F. Baker, and B. VerSteeg, “PIE: A lightweight control scheme to address the bufferbloat problem,” in *Proc. IEEE HPSR*, 2013.
- [4] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [5] H. van den Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. AAAI*, 2016.
- [6] M. Kim, “Deep reinforcement learning based active queue management for IoT networks,” *Journal of Network and Systems Management*, 2019.
- [7] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell, and J. Kopena, “Network simulations with the ns-3 simulator,” *SIGCOMM Demo*, 2008.
- [8] Texas Instruments, “CC2420 2.4 GHz IEEE 802.15.4 / ZigBee-ready RF transceiver datasheet,” 2007. [Online]. Available: <https://www.ti.com/product/CC2420>
- [9] tiny-dnn contributors, “tiny-dnn: A header only, dependency-free deep learning framework in C++14,” v1.0.0a3, 2016. [Online]. Available: <https://github.com/tiny-dnn/tiny-dnn>